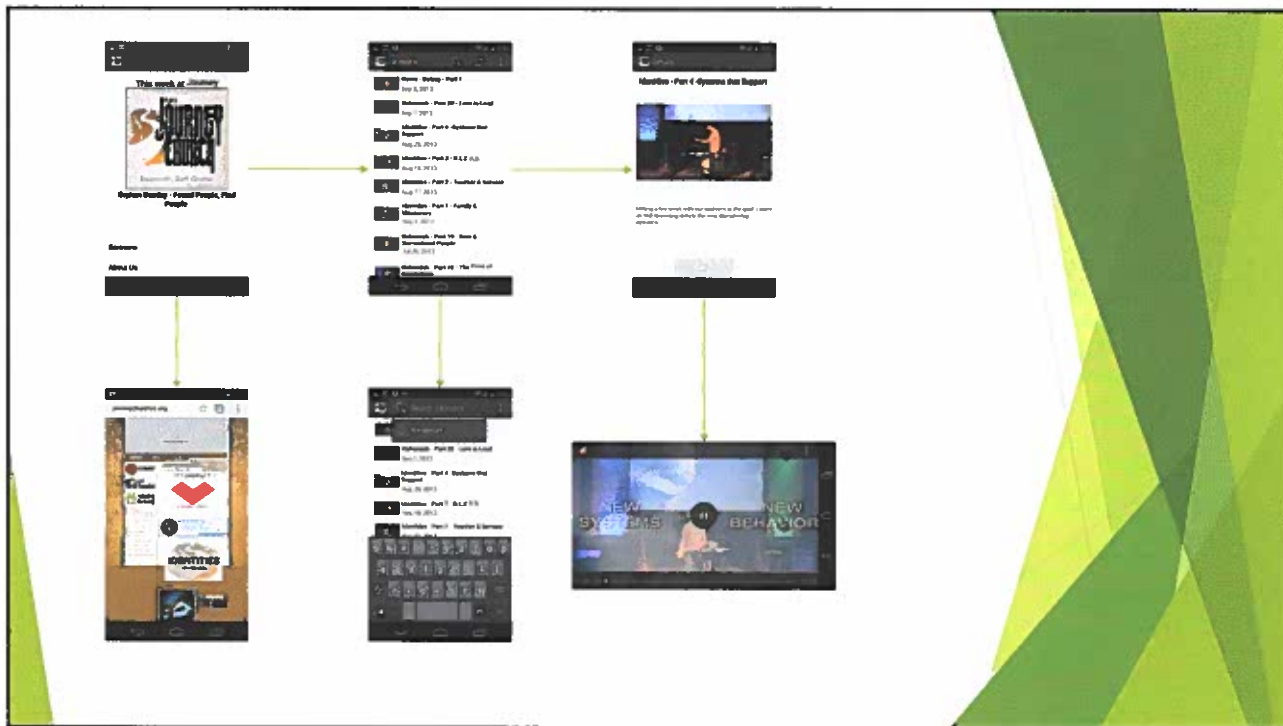


Senior Project

Android App for Churches

Concept

- ▶ Create an app to allow church members to stay up-to-date with the church
- ▶ Allow users to watch the sermons wherever they are
 - ▶ Keep track of which sermons have been watched
 - ▶ Allow users to search for a past sermon
 - ▶ Provide a summary of each sermon
- ▶ Provide user with information about the church



Welcome Screen

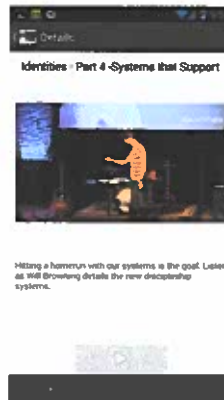
- Displays the current sermon from the church
- Sermon can be touched and it will take you to the details for that sermon
- Selecting "Sermons" takes you to a list of all the sermons
- Selecting "About Us" takes you to the churches website



Sermons



- Lists all sermons in database with the thumbnail for each video
- When sermons are selected, it shows the title, thumbnail, and description of the sermon
- The videos of the sermons can be streamed from the server



Sermon Layouts

```

1  xmlns:tools="http://schemas.android.com/tools"
2  android:layout_width="match_parent"
3  android:layout_height="match_parent"
4  android:layout_marginLeft="16dp"
5  android:layout_marginRight="16dp"
6  android:baselineAligned="false"
7  android:divider="?android:attr/dividerVertical"
8  android:orientation="vertical"
9  android:showDividers="middle"
10 tools:context=".activities.SermonListActivity"
11 tools:layout="@layout/sermon_list_twopane"
12
13 <fragment android:id="@+id/sermon_detail_container"
14     android:layout_height="0dp"
15     android:name="com.chris.churchapp.fragments.SermonDetailFragment"
16     android:layout_weight="1"
17     android:layout_width="match_parent"
18     tools:layout="@layout/fragment_sermon_detail" />
19
20 <fragment
21     android:id="@+id/sermon_list"
22     android:name="com.chris.churchapp.fragments.SermonListFragment"
23     android:layout_width="wrap_content"
24     android:layout_height="0dp"
25     android:layout_weight="1"
26     tools:layout="@layout/activity_sermon_list"
27     android:layout_gravity="bottom" />
28
29 </include>

```



Search



```

108 private static String QUERY_EXTRA_KEY = "QUERY_EXTRA_KEY";
109
110
111 private void parseIntent(Intent intent) {
112     // If the Activity was started to service a Search request,
113     // extract the search query.
114     if (intent.getAction().equals(Intent.ACTION_SEARCH)) {
115         String searchQuery = intent.getStringExtra(SearchManager.QUERY);
116
117         //Setup Recent Suggestions
118         SearchRecentSuggestions suggestions = new SearchRecentSuggestions(
119             this,
120             SermonSuggestionProvider.AUTHORITY,
121             SermonSuggestionProvider.MODE);
122
123         suggestions.saveRecentQuery(searchQuery, null);
124
125         // Perform the search, passing in the search query as an argument
126         // To the Cursor Loader
127         Bundle args = new Bundle();
128         args.putString(QUERY_EXTRA_KEY, searchQuery);
129
130         // Restart the Cursor Loader to execute the new Query.
131         getLoaderManager().restartLoader(0, args, this);
132     }
133 }
134

```

What's Next



Charleston Southern University

Senior Project Report

Church App

Chris Gilbert
11/22/2013

Contents

Senior Project Report	2
Background.....	2
Problem Description.....	2
Proposed Solution	5
Implementation.....	6
Layouts and UI components.....	6
Implementing Activities and Fragments	10
Downloading the Sermon List	13
SQLite Database	15
Results	23

Senior Project Report

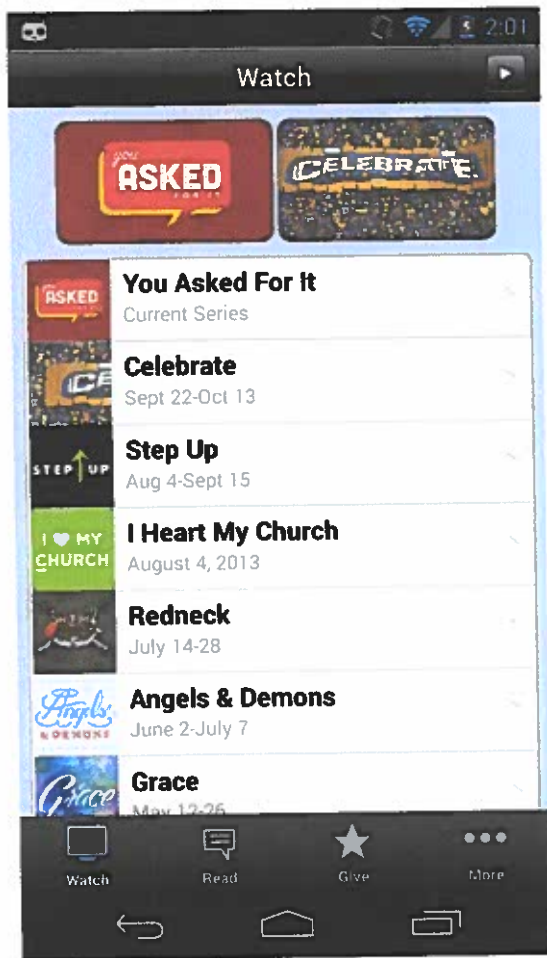
Background

I have been interested in smart phone since the Motorola Droid was released on October 17, 2009. This was the first Android device that operated on the Verizon network. While it still had the app store, the number and quality of the apps available at the time was significantly smaller. As the devices are becoming more and more popular, the app market began to take off. Google announced at their Q2 2013 earning call that there have been more than 50 billion app downloads. With increase in applications, it is usually easy to find an app to do most things people would expect from a smart phone. While most of these apps have significantly improved over time, I found that there was yet to be an app for churches that really stood out.

Problem Description

There are many churches that have apps for Android devices, but most of them that I have seen were poorly written and provided a poor user experience. These claims come from the fact that they apps are constantly force closing and a lot of them are using layouts that were clearly not designed for the Android operating system.

A great example of these types of apps are the apps made by a company called Subsplash. If you do a search for "church apps" in the Google Play Store, you will see that most of the apps at the top of the list are made by Subsplash. According to Subsplash, "The Subsplash Platform / The Church App is a mobile app platform built by Subsplash that enables you to create high-quality, highly-configurable apps for your church, ministry, or organization. The result is a suite of rich and powerful mobile apps for iPhone, iPad, Android Phone, Windows Phone, and the web". A screenshot of an app from Subsplash can be seen below.



While the app does appear to be functional on this device, it is obvious that the app was not originally designed for the Android operating system. This can cause many problems. One problem is that the app can be very confusing to new users. This app is designed with the navigation principles of iOS in mind. It does not follow any of the guidelines Google has recommended for effective navigation on an Android device. Another issue with apps created like this is that they do not get to take advantage of all the benefits of the software development kit. The Android operating system has gone through many changes over its relatively short life. This has caused many changes in the software development kit. The more an app deviates from the standard design principles, the harder it is for it to adapt to the changes in the software development kit. An example of the issues this can cause can be seen in the

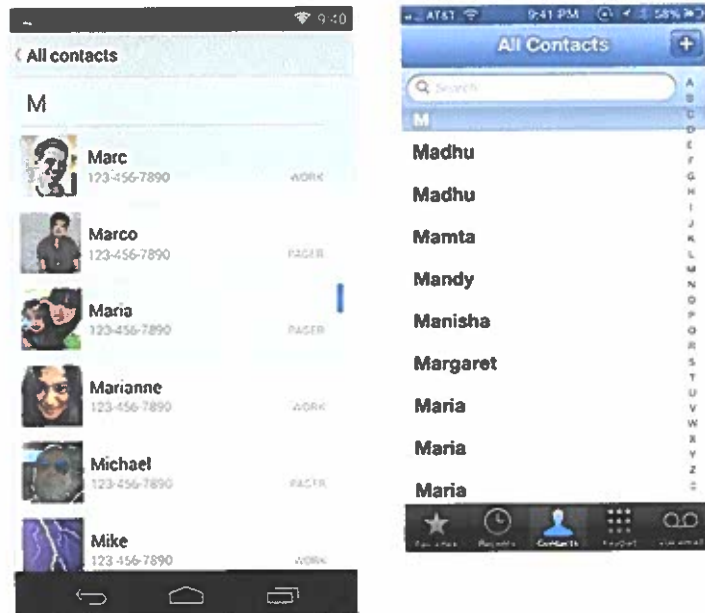
screenshot below. The second image was found on the Android developer website while developing this app. It helps explain the issues also.

Don't use bottom tab bars

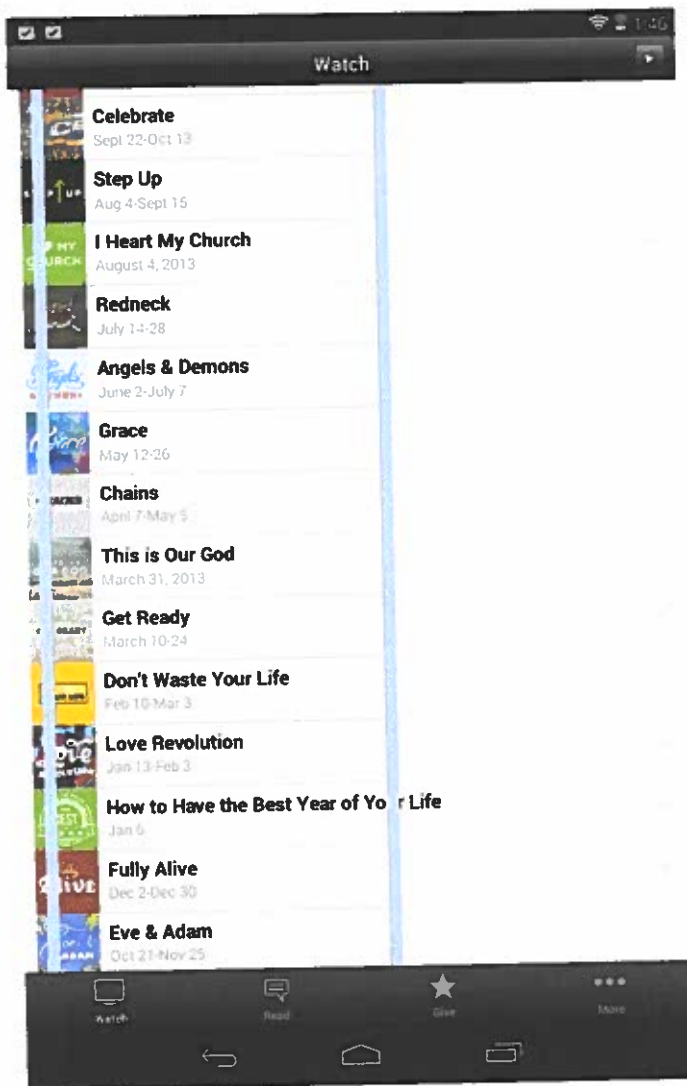
Other platforms use the bottom tab bar to switch between the app's views. Per platform convention, Android's tabs for view control are shown in action bars at the top of the screen instead. In addition, Android apps may use a bottom bar to display actions on a split action bar.

You should follow this guideline to create a consistent experience with other apps on the Android platform and to avoid confusion between actions and view switching on Android.

For more information on how to properly use action bars for view control, see [Action Bars](#).



Android dialer with tabs in an action bar vs. bottom tabs in iOS.



This screenshot was taken after opening the app on a Nexus 7 tablet. This device has a 7-inch screen. As you can see, the layout does not scale causing the app to become unusable on this device. While I do not know everything Subsplash offers, their website to sign a church up to use this app says "Packages starting at \$499". This is a significant amount of money considering the issues with this app.

Proposed Solution

The solution I propose is to create an app from the ground up for Android devices that follows the guidelines provided by Google for a good app. The app will store information about each sermon in a SQLite database. This will allow the app to quickly show the list of sermons even when the network

connection is slow. It will also allow user specific information to be stored about each sermon, such as whether the sermon has been view or not. The database will store everything about each sermon except for the actual thumbnail image. It will save the URL for the image and will download it as the network connection allows. This will save a lot of space on the device by not storing all the images. Storing the data for each sermon in a SQLite database will also make it easy to implement the ability to search the sermons.

```
<item>
  <title>Nehemiah - Relapses &amp; Neglect - Part 23 (v)</title>
  <pubDate>Mon, 28 Oct 2013 01:00:00 +0000</pubDate>
  <guid isPermaLink="false"><![CDATA[8e766f04e74"b0"e0883a52d273c3410]]></guid>
  <link><![CDATA[https://journeymedia.libsyn.com/nehemiah-relapses-neglect-part-23-v]]></link>
  <media:thumbnail url="http://assets.libsyn.com/item/2527795" />
  <description><![CDATA[Nehemiah 13:15-31 In this final sermon on Nehemiah Pastor Will Browning reminds us that we must continue forward with God and not rest on the things in the past.]]></description>
  <enclosure length="175661541" type="video/x-mp4" url="http://traffic.libsyn.com/journeymedia/20131027_Nehemiah_Part_23_itunes.mp4" />
  <itunes:duration>39:39</itunes:duration>
  <itunes:explicit>no</itunes:explicit>
  <itunes:keywords>browning, sermon, will, nehemiah, neglect, journeychurchsc, relapses</itunes:keywords>
  <itunes:subtitle><![CDATA[Nehemiah 13:15-31 In this final sermon on Nehemiah Pastor Will Browning reminds us that we must continue forward with God and not rest on the things in the past.]]></itunes:subtitle>
</item>
```

The sermons will displayed in a list that shows the user the title of the sermon and the date the sermon was recorded. When the sermon is selected, the app will show the details about that specific sermon and allow the user to stream the video.

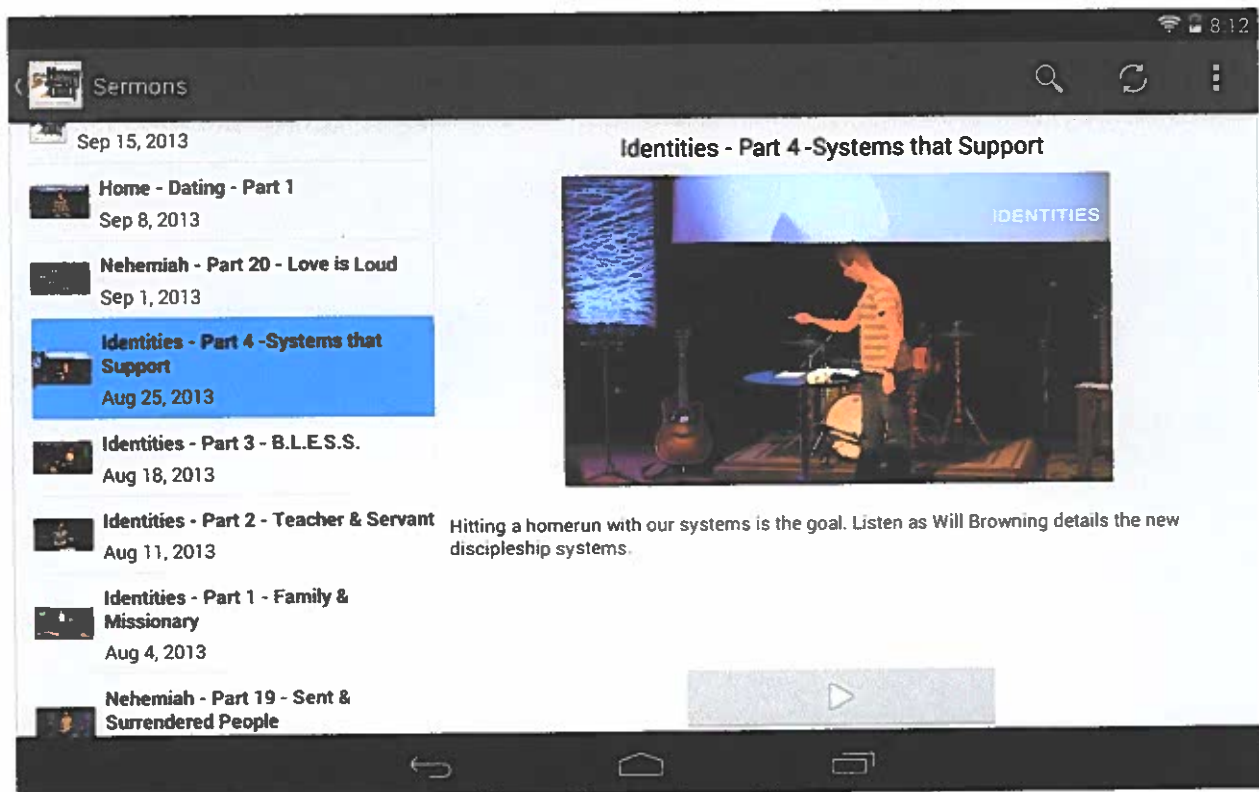
Implementation

Layouts and UI components

The app uses several activities to provide effective navigation in the app. An activity is a single screen in the app that has a user interface. The activities in my app are the welcome screen, the sermon list, the sermon details, and the activity that is used for displaying search results. Each activity has a lifecycle that is managed by the system. Android apps must implement the methods necessary to ensure the app behaves correctly at each stage in the lifecycle. These methods can be seen in the image from the android development website that is added below.

The app uses another major user interface component called a fragment. Fragments can be thought of “as a modular section of an activity, which has its own lifecycle, receives its own input events, and which you can add or remove while the activity is running”

(<http://developer.android.com/guide/components/fragments.html>). Fragments were designed “to support more dynamic and flexible UI designs on large screens, such as tablets”. This app is using two fragments, as seen in the image below.



One of the fragments is the list of sermons on the left. The other fragment is the details of the sermon on the right. The list of sermons can be scrolled independently. Each sermon also has its own lifecycle. If the app were not using fragments, the list of sermons on the left would span across the screen. This would be a waste of space because most of the sermons would not use even half of the screen.

Another technique used to ensure that the user gets the best experience possible is the use of multiple layouts for portrait and landscape. The arrangement of the fragments seen above is only used on devices with screens that are 7 inches or larger when they are in landscape mode. These fragments can also be rearranged while the app is running. An example of this can be seen in the screenshot below.



Identities - Part 4-Systems that Support



Hitting a homerun with our systems is the goal. Listen as Will Browning details the new discipleship systems.



Home - Dating Before Marriage - Part 2

Sep 15, 2013



Home - Dating - Part 1

Sep 8, 2013



Nehemiah - Part 20 - Love is Loud

Sep 1, 2013



Identities - Part 4-Systems that Support

Aug 25, 2013



Identities - Part 3 - B.L.E.S.S.

Aug 18, 2013



Identities - Part 2 - Teacher & Servant

Aug 11, 2013



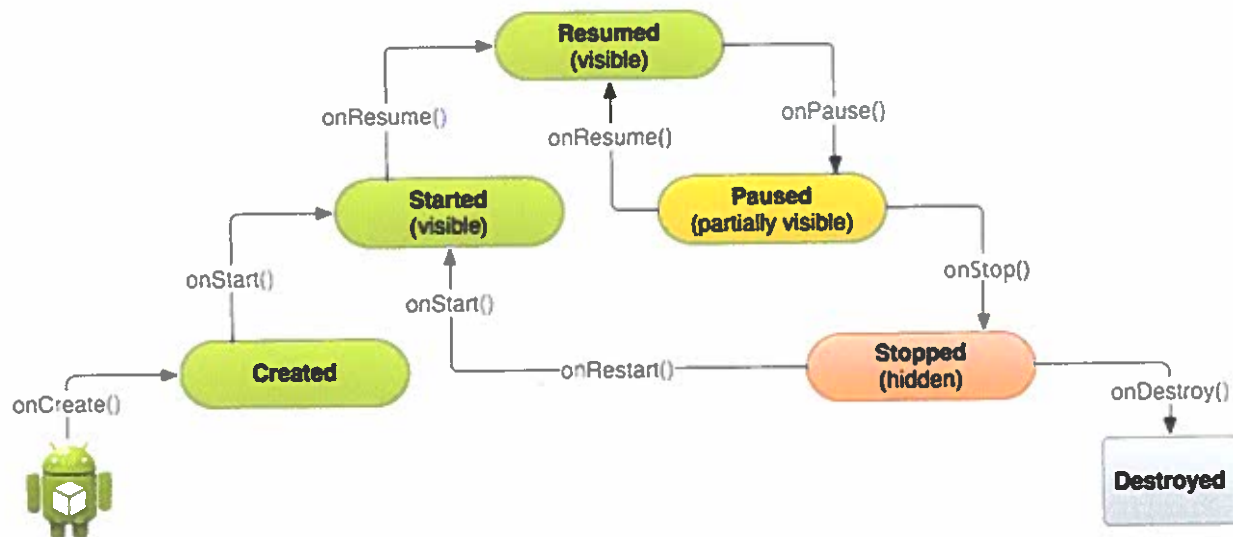
Identities - Part 1 - Family & Missionary

Aug 4, 2013



Implementing Activities and Fragments

Because of the limitations of mobile devices, the Android operating system must handle the lifecycle much more actively than most are probably used to. This is done to deliver a good experience and to allow multitasking on devices with relatively small amounts of memory and power limitations. When implementing Activities and Fragments, special care must be taken to make sure the code is ran in the order it was intended. This very frustrating at first because the app would constantly force close with null pointer errors. Once implemented correctly, these methods help ensure the app behaves as expected through all the situations a user will experience while using the device. Some of the situations that could cause an app with these methods implanted improperly misbehave include receiving a phone call while using the app, switching to another app, returning to the app, and rotating the device. While force closing is the most obvious sign of this, it could also cause the app to consume valuable system resources that the user is not benefiting from. A



An example of an implanted method form my SermonListActivity can be seen below in the screenshot.

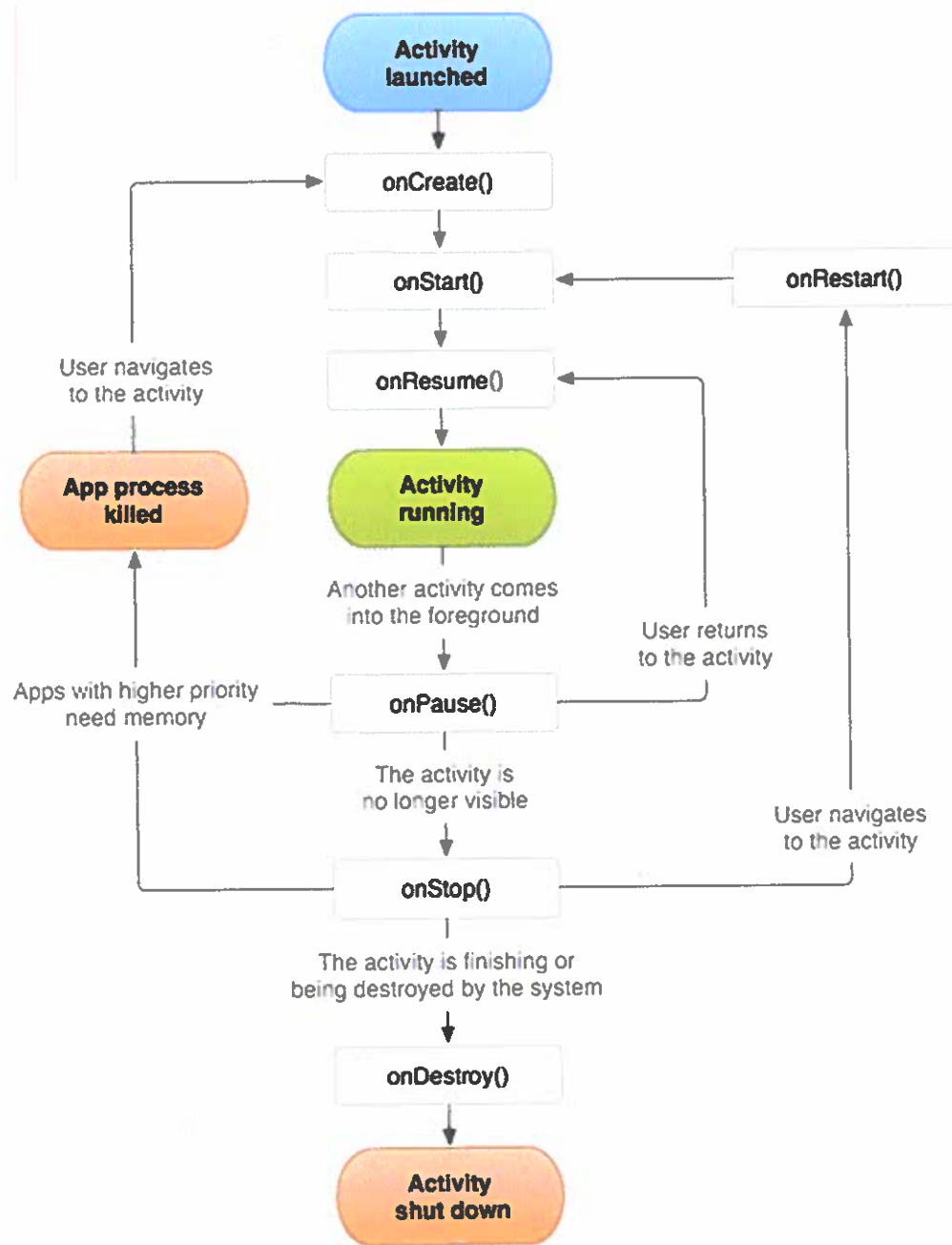
```

40
41  @Override
42  protected void onCreate(Bundle savedInstanceState) {
43      super.onCreate(savedInstanceState);
44      Log.i(TAG, "onCreate");
45      setContentView(R.layout.activity_sermon_list);
46
47      final ActionBar actionBar = getActionBar();
48      actionBar.setDisplayHomeAsUpEnabled(true);
49
50      // find our fragments
51      mSermonListFragment = (SermonListFragment) getFragmentManager().findFragmentById(
52          R.id.sermon_list);
53      mSermonDetailFragment = (SermonDetailFragment) getFragmentManager().findFragmentById(
54          R.id.sermon_detail_container);
55
56      mSermonListFragment.setOnSermonSelectedListener(this);
57
58      // Determine whether we are in single-pane or dual-pane mode by testing the visibility
59      // of the article view.
60      View SermonDetailsView = findViewById(R.id.sermon_detail_container);
61      mIsDualPane = SermonDetailsView != null && SermonDetailsView.getVisibility() == View.VISIBLE;
62
63      mSermonListFragment.setActivateOnItemClick(mIsDualPane);
64      restoreSelection(savedInstanceState);
65
66  }

```

This implementation is interesting for a few reasons. This method is to perform basic application startup logic that should happen only once for the entire life of the activity. All of this is happening before the app even becomes visible to the user. You can see at line 45 the app is getting the layout resource (defined in a XML file) so it can begin setting up the layout. The next two lines of code setup the Action bar. This is the bar that goes across the top of Android app that, among other things, provides the menu options that used to require a dedicated hardware button. The next couple of lines is where it gets even more interesting. These lines of code are looking for the resource id for the fragments. Because every fragment in Android must live inside of an activity, this activity begins setting up the SermonListFragment. This is the part that contains the list of all the sermons. The next fragment, the SermonDetailFragment, only is created if the screen on the device is big enough to show these two fragments side by side. This app is setup to only allow them to be placed side-by-side on devices with screens seven inches or larger. The method then looks for the fragment using the id (line 60). Line 61 then checks to see if it found the SermonDetailsView. If the view is found it will set mIsDualPane to true.

This variable is used in the rest of the activity to determine how to arrange the fragments when they need to be swapped.



Downloading the Sermon List

The sermons are stored on the server in an XML file on the server. The app downloads the file, parses it, and stores each sermon “item” in a SQLite database.

Connecting to the network on an Android device must be handled carefully. There are multiple ways for these devices to get a network connection. The app must make sure there is a network connection before trying to download the XML file. Android provides two classes, `ConnectivityManager` and `NetworkInfo`, to help check the network connection.

```
30
31     ConnectivityManager connMgr = (ConnectivityManager)
32         getSystemService(Context.CONNECTIVITY_SERVICE);
33     NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();
34     if (networkInfo != null && networkInfo.isConnected()) {
35         // fetch data
36         Intent refreshIntent = new Intent(getApplicationContext(), RefreshService.class);
37         startService(refreshIntent);
38     } else {
39         // show error
40         Toast.makeText(
41             this,
42             "Unable to load content. Check your network connection.",
43             Toast.LENGTH_LONG).show();
44     }
45 }
```

Because network connections can be unreliable, it could potentially take a while to download the XML file. If this was done on the UI thread, the app could become unresponsive and force close. One way to handle this is to use a type of service called an `IntentService`. An `IntentService` is a service that handles asynchronous requests on demand and stops itself when it runs out of work. It handles requests on a worker thread that will run as long as necessary without blocking the applications main loop. An example of the app starting the `IntentService` is on line 37 in the image above. When the `IntentService` is started, the `onHandleIntent` method is called. If the service has many uses, the method could be set up to do different things based on the intent received. The `refreshService` in this app is set up for one

purpose so this was not necessary. The implementation of the `onHandleIntent` used by the `refreshService` can be seen below.

```
38
39      @Override
40      protected void onHandleIntent(Intent intent) {
41          Log.v(TAG, "onHandleIntent");
42          try {
43              loadXmlFromNetwork("http://journeymedia.libsyn.com/rssvideo");
44          } catch (IOException e) {
45              "Unable to load content. Check your network connection.";
46          } catch (XmlPullParserException e) {
47              "Error parsing XML.";
48          }
49      }
```

This must use a try/catch method because the network connection is unreliable. It then calls `loadXmlFromNetwork` and if everything works correctly adds the sermons to the database. If something goes wrong, it will catch the error depending on what went wrong. The `loadXmlFromNetwork` method passes the URL of the XML file to the `downloadUrl` method. The `downloadUrl` method returns an `InputStream` of the XML file. The `InputStream` is then passed to the parser for the church. This could easily be modified to work with a different church in the future. The app then loops through each of the items (sermons) and adds them to the database starting at line 98 in the image below.

```

86
87 public String loadXmlFromNetwork(String urlString) throws XmlPullParserException, IOException {
88     InputStream stream = null;
89     JourneyChurchXmlParser journeyChurchXmlParser = new JourneyChurchXmlParser();
90     //items = null;
91
92     try{
93         stream = downloadUrl(urlString);
94         items = journeyChurchXmlParser.parse(stream);
95
96         //test adding items to SQL
97         Iterator<SermonItem> it = items.iterator();
98         while(it.hasNext()) {
99             //sermonData.insert(it.next());
100             SermonItem temp = it.next();
101             addNewSermon(temp);
102         }
103
104         // make sure input stream is closed after app is done with it
105     } finally{
106         if(stream != null){
107             stream.close();
108         }
109     }
110     return null;
111 }
112
113 // Given a string representation of a URL, sets up a connection and gets
114 // an input stream.
115 private InputStream downloadUrl(String urlString) throws IOException {
116     URL url = new URL(urlString);
117     HttpURLConnection conn = (HttpURLConnection) url.openConnection();
118     conn.setReadTimeout(10000 /* milliseconds */);
119     conn.setConnectTimeout(15000 /* milliseconds */);
120     conn.setRequestMethod("GET");
121     conn.setDoInput(true);
122     // Starts the query
123     conn.connect();
124     InputStream stream = conn.getInputStream();
125     return stream;
126 }
127

```

SQLite Database

After parsing the XML file and getting the list of sermons, they must be stored immediately to ensure the data is saved before the user navigates away from the app or the system closes the app to use the ram for something else. Android allows you to store data using key-value sets, files, or SQLite databases. Each app can have its own databases that are inaccessible to other apps by default. This allows the app to provide quick searches and the ability to store information about each sermon, such as whether it has been watched or not. Another reason to use a database is that the Android SDK provides

a lot of tools and adapters to make things easier once the database is set up. It will also allow you to add features such as a widget easier in the future.

Creating the database

The first step in setting up an app for a SQLite database is to create a contract class. "A contract class is a container for constants that define names for URIs, tables, and columns. The contract class allows you to use the same constants across all the other classes in the same package." The contract class for this app is called `SermonsContract` and can be seen below.

```

1 package com.chris.churchapp.database;
2
3 import android.net.Uri;
4 import android.provider.BaseColumns;
5
6 public final class SermonsContract {
7     public static final String AUTHORITY = "com.chris.churchapp.sermonprovider";
8
9     public static final String DB_NAME = "sermons.db";
10    public static final int DB_VERSION = 1;
11    public static final String TABLE = "sermons";
12    public static final String SORT_ORDER = Columns.PUBDATE + " desc";
13
14    //This class cannot be instantiated
15    private SermonsContract() {}
16
17    public static final class Columns implements BaseColumns {
18        private Columns() {}
19
20        /**
21         * 0-relative position of a note ID segment in the path part of a note ID URI
22         */
23        public static final int SERMON_ID_PATH_POSITION = 1;
24
25        /**
26         * The scheme part for this provider's URI
27         */
28        private static final String SCHEME = "content://";
29
30        /**
31         * Path part for the Sermons URI
32         */
33        private static final String PATH_SERMONS = "/sermons";
34
35        /**
36         * Path part for the Note ID URI
37         */
38        private static final String PATH_SERMON_ID = "/sermons/";
39
40        /**
41         * The content:// style URL for this table
42         */
43        public static final Uri CONTENT_URI = Uri.parse(SCHEME + AUTHORITY + PATH_SERMONS);
44
45        /**
46         * The content URI base for a single note. Callers must
47         * append a numeric note id to this Uri to retrieve a note
48         */
49        public static final Uri CONTENT_ID_URI_BASE
50            = Uri.parse(SCHEME + AUTHORITY + PATH_SERMON_ID);
51
52        public static final String TITLE = "sermon_title";
53        public static final String PUBDATE = "sermon_pubdate";
54        public static final String DESCRIPTION = "sermon_description";
55        public static final String URL = "sermon_video_url";
56        public static final String THUMBNAIL_URL = "sermon_video_thumbnail_url";
57        public static final String WATCHED_STATE = "watched_state";
58    }
59 }
60

```


To create the SQLite database, android provides a set of APIs in the SQLiteOpenHelper class. The DBHelper class in this app extends the SQLiteOpenHelper and overrides the onCreate and onUpgrade methods. The database is created using execSQL(DB_CREATE) on line 32 in the image below. This create a SQLite database with a table called “sermons”. The table has columns for the sermon id, title, publication date, sermon description, watched state, URL for the video, and URL for the thumbnail. The full implementation of the SQLiteOpenHelper class can be seen below

```

1  package com.chris.churchapp.database;
2
3  import ...
4
5
6
7
8  public class DbHelper extends SQLiteOpenHelper {
9
10     static final String TAG = "DbHelper";
11     private static final String DB_CREATE =
12         "create table " + SermonsContract.TABLE + " ("
13         + SermonsContract.Columns._ID + " integer primary key autoincrement, "
14         + SermonsContract.Columns.TITLE + " text not null, "
15         + SermonsContract.Columns.PUBDATE + " INTEGER, "
16         + SermonsContract.Columns.DESCRPTION + " text, "
17         + SermonsContract.Columns.WATCHED_STATE + " INTEGER, "
18         + SermonsContract.Columns.URL + " text not null, "
19         + SermonsContract.Columns.THUMBNAIL_URL + " text);";
20
21     // Constructor
22     public DbHelper(Context context) {
23         super(context, SermonsContract.DB_NAME, null, SermonsContract.DB_VERSION);
24     }
25
26     // Called only once, first time the DB is created
27     @Override
28     public void onCreate(SQLiteDatabase db) {
29
30         Log.d(TAG, "onCreated sql: " + DB_CREATE);
31
32         db.execSQL(DB_CREATE);
33     }
34
35     // Called whenever newVersion != oldVersion
36     @Override
37     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
38         // Typically do ALTER TABLE statements, but...we're just in development,
39         // so:
40
41         db.execSQL("drop table if exists " + SermonsContract.TABLE); // remove the old database
42         // away
43         Log.d(TAG, "onUpdated");
44         onCreate(db); // run onCreate to get new database
45     }
46
47 }

```

Content provider

The Android SDK allows creation of something called a content provider. "A content provider manages access and creation to a central repository of data". This app did not require a content provider for the sermons database, but setting the app up this way now will allow easy implementation of an application widget in the future. The app also uses a content provider to provide the recent search

history. A method from this app that is used to access the content provider providing the recent search history can be seen below in the screenshot.

```
108
109     private static String QUERY_EXTRA_KEY = "QUERY_EXTRA_KEY";
110
111     private void parseIntent(Intent intent) {
112         // If the Activity was started to service a Search request,
113         // extract the search query.
114         if (Intent.ACTION_SEARCH.equals(intent.getAction())) {
115             String searchQuery = intent.getStringExtra(SearchManager.QUERY);
116
117             //Setup Recent Suggestions
118             SearchRecentSuggestions suggestions = new SearchRecentSuggestions(
119                 this,
120                 SermonSuggestionProvider.AUTHORITY,
121                 SermonSuggestionProvider.MODE);
122
123             suggestions.saveRecentQuery(searchQuery, null);
124
125             // Perform the search, passing in the search query as an argument
126             // to the Cursor Loader
127             Bundle args = new Bundle();
128             args.putString(QUERY_EXTRA_KEY, searchQuery);
129
130             // Restart the Cursor Loader to execute the new query.
131             getLoaderManager().restartLoader(0, args, this);
132         }
133     }
134
```

Cursor Loaders

Because I chose to implement a content provider in my app, there are now many powerful features in the SDK that are available for us to use. An example of this is an extension of the Loader class call the CursorLoader. Because querying the database can potentially take a while, especially when the network is involved, it should never be performed on the apps user interface thread (the main thread). The CursorLoader class can help solve the issue by allowing the app to query the ContentResolver and returning a Cursor. This class makes it significantly easier query cursors using an AsyncTaskLoader. This

means that the query is put on a different thread than the user interface thread so that it cannot slow it down to the point that it force closes. An example of this can be seen in the screenshot below.

```
81
82     @Override
83     public void onCreate(Bundle savedInstanceState) {
84         super.onCreate(savedInstanceState);
85         Log.i(TAG, "onCreate");
86
87         simpleCursorAdapter = new SimpleCursorAdapter(
88             getActivity(),
89             R.layout.sermon_list_item,
90             null,
91             FROM,
92             TO,
93             0
94         );
95
96         simpleCursorAdapter.setViewBinder(new SermonViewBinder());
97         setListAdapter(simpleCursorAdapter);
98
99     }
```

```

120
121     @Override
122     public void onActivityCreated(Bundle savedInstanceState) {
123         super.onActivityCreated(savedInstanceState);
124         Log.i(TAG, "onActivityCreated");
125
126         setEmptyText("No sermons found");
127         getLoaderManager().initLoader(0, null, this);
128
129     }
130
131
241
242     @Override
243     public Loader<Cursor> onCreateLoader(int id, Bundle args) {
244         Log.i(TAG, "onCreateLoader");
245
246         return new CursorLoader(getActivity(), SermonsContract.Columns.CONTENT_URI,
247             null, null, null, null);
248     }
249
250     @Override
251     public void onLoadFinished(Loader<Cursor> arg0, Cursor c) {
252         Log.i(TAG, "onLoadFinished");
253
254         simpleCursorAdapter.swapCursor(c);
255
256     }
257
258     @Override
259     public void onLoaderReset(Loader<Cursor> arg0) {
260         Log.i(TAG, "onLoaderReset");
261         simpleCursorAdapter.swapCursor(null);
262     }

```

The methods `onCreateLoader`, `onLoadFinished`, and `onLoaderReset` are required methods for a `CursorLoader`. With the help of the content provider, they be implemented very quickly. Another feature of Loaders is that they monitor the source of their data and deliver new results when the content changes. They also automatically reconnect to the last loader's cursor when being recreated after a configuration change. This is good because it means that the app does not have to waste resources trying to query the data again. The loader is started with the `getLoaderManger().initLoader` function at line 127. When it is done querying the data on a different thread, it swaps the cursor into the adapter that the app has that is setup to load the data into the custom list View items for the sermons.

Results

The app turned out to be a powerful tool that should be able to be modified easily to work with other churches. It follows the android guidelines to provide a fluid and effective user interface that should allow church members watch sermons wherever they are. Because the app does not store the images with the regular device storage, it allows members to install the app even on devices with limited storage space without wasting space. The app installed on my Galaxy Nexus is only using 1.59 MB of the devices app storage. The app is using 10.9 MB of cache for the images to provide smooth scrolling and to keep the app from downloading the images every time the app starts which would greatly affect the performance and waste the user's data plan. This also allows the system to remove the cached data if the space is needed for something else.

One thing that I was currently unable to implement was a news feed for the church. While I did have a Twitter feed set up that provided all the Tweets tagged with the name of the church, their API has since changed and now requires authentication. Since I am not working directly with a church at the moment, I chose to remove it for now. When the app is being set up to work with a church it will not be difficult to add a news feed especially if it can be download as an XML file like the sermon information.

Another thing that still needs to be worked on is handling the links to the sermons that are in the database that are no longer available. This also should not be hard to add, I just needed more details about how they run their servers and how long they leave the videos on their server so I can implement it. This would also be best to do when setting up for the church so that it will be specific to how they have their server set up.

Works Cited

"The Church App - Create a High Quality App for Your Church or Ministry." *The Church App - Create a High Quality App for Your Church or Ministry*. Subsplash, n.d. Web. 22 Nov. 2013. <<http://www.thechurchapp.org/>>.

Meier, Reto. *Professional Android 4 Application Development*. 3rd ed. Indianapolis: John Wiley & Sons, 2012. Print.

"Reference." *Android Developers*. N.p., n.d. Web. 21 Nov. 2013. <<http://developer.android.com/reference/packages.html>>.

```

1 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
2   package="com.chris.churchapp"
3   android:versionCode="1"
4   android:versionName="1.0">
5
6   <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="16" />
7
8   <uses-permission android:name="android.permission.INTERNET" />
9   <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
10  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
11
12  <application android:allowBackup="true"
13     android:name="com.chris.churchapp.ChurchApp"
14     android:icon="@drawable/ic_launcher_journey"
15     android:label="@string/app_name"
16     android:theme="@style/AppTheme">
17     <provider android:name=".database.SermonSuggestionProvider"
18        android:authorities="com.chris.churchapp.database.SermonSuggestionProvider"
19        android:enabled="true"
20        android:exported="false"/>
21     <activity
22        android:name=".activities.MainActivity"
23        android:label="@string/app_name">
24        <intent-filter>
25            <action android:name="android.intent.action.MAIN" />
26            <category android:name="android.intent.category.LAUNCHER" />
27        </intent-filter>
28    </activity>
29    <activity
30        android:name=".activities.SermonListActivity"
31        android:label="Sermons">
32        <meta-data android:name="android.app.default_searchable"
33            android:value=".activities.SermonSearchActivity"/>
34    </activity>
35    <activity
36        android:name=".activities.SermonDetailActivity"
37        android:label="Details">
38        <!--android:parentActivityName=".SermonListActivity"-->
39        <!--&lt;!&ndash; Parent activity meta-data to support 4.0 and lower &ndash;&gt;-->
40        <!--<meta-data-->
41        <!--android:name="android.support.PARENT_ACTIVITY"-->
42        <!--android:value=".SermonListActivity"/>-->
43    </activity>
44    <activity android:name=".activities.SermonSearchActivity"
45        android:launchMode="singleTop">
46        <intent-filter>
47            <action android:name="android.intent.action.SEARCH" />
48        </intent-filter>
49        <meta-data android:name="android.app.searchable"

```



```
50     android:resource="@xml/searchable"/>
51 </activity>
52 <service
53     android:name="com.chris.churchapp.RefreshService"
54     android:enabled="true"
55     android:exported="false" >
56 </service>
57
58 <provider
59     android:name="com.chris.churchapp.database.SermonProvider"
60     android:authorities="com.chris.churchapp.database.SermonProvider"
61     android:enabled="true"
62     android:exported="false" >
63 </provider>
64
65 </application>
66
67 </manifest>
68
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2
3 <searchable xmlns:android="http://schemas.android.com/apk/res/android"
4     android:label="@string/app_name"
5     android:hint="@string/sermon_search_instructions"
6     android:searchSuggestAuthority="com.chris.churchapp.database.SermonSuggestionProvider"
7     android:searchSuggestSelection="?">
8 </searchable>
```

```
1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2
3 <item
4     android:id="@+id/menu_search"
5     android:title="@string/menu_search"
6     android:showAsAction="ifRoom|collapseActionView"
7     android:icon="@drawable/ic_menu_search"
8     android:actionViewClass="android.widget.SearchView">
9 </item>
10 <item
11     android:id="@+id/menu_refresh"
12     android:title="@string/menu_refresh"
13     android:showAsAction="ifRoom"
14     android:icon="@drawable/ic_menu_refresh">
15 </item>
16 <item android:id="@+id/menu_settings"
17     android:title="@string/menu_settings"
18     android:orderInCategory="100"
19     android:showAsAction="never" />
20 </menu>
21
```

```
1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2
3   <item
4       android:id="@+id/action_settings"
5       android:orderInCategory="100"
6       android:showAsAction="never"
7       android:title="@string/action_settings"/>
8
9 </menu>
```

```
1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2   <item android:id="@+id/menu_settings"
3       android:title="@string/menu_settings"
4       android:orderInCategory="100"
5       android:showAsAction="never" />
6 </menu>
7
```

```
1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2   <item android:id="@+id/menu_settings"
3       android:title="@string/menu_settings"
4       android:orderInCategory="100"
5       android:showAsAction="never" />
6 </menu>
7
```

```

1 <?xml version="1.0" encoding="utf-8"?>
2
3 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:orientation="vertical"
6     android:layout_width="fill_parent"
7     android:layout_height="fill_parent">
8
9     <LinearLayout
10         android:orientation="vertical"
11         android:layout_width="fill_parent"
12         android:layout_height="0dp"
13         android:layout_weight="2"
14         android:clickable="true"
15         android:id="@+id/sermon_linear_layout">
16
17         <TextView
18             android:layout_width="wrap_content"
19             android:layout_height="wrap_content"
20             android:textAppearance="?android:attr/textAppearanceLarge"
21             android:text="This week at Journey"
22             android:layout_gravity="center"
23             android:gravity="center"
24             android:layout_alignParentTop="true"
25             android:layout_alignParentLeft="true"
26             android:layout_alignParentRight="true"
27             android:visibility="visible"
28             android:paddingTop="15dp"/>
29
30         <ImageView
31             android:layout_width="match_parent"
32             android:layout_height="0dp"
33             android:id="@+id/imageView"
34             android:src="@drawable/journey_logo"
35             android:scaleType="fitCenter"
36             android:layout_below="@+id/main_title_textview"
37             android:layout_above="@+id/main_sermon_title_textView"
38             android:layout_centerHorizontal="true"
39             android:paddingLeft="10dp"
40             android:paddingRight="10dp"
41             android:adjustViewBounds="false"
42             android:visibility="gone"
43             android:layout_weight="2"/>
44
45         <TextView
46             android:layout_width="wrap_content"
47             android:layout_height="wrap_content"
48             android:textAppearance="?android:attr/textAppearanceMedium"
49             android:id="@+id/main_sermon_title_textView"

```

```

50     android:layout_gravity="center"
51     android:textStyle="bold"
52     android:gravity="center"
53     android:text="Welcome"
54     android:layout_above="@+id/main_listView"
55     android:layout_centerHorizontal="true"
56     android:paddingLeft="10dp"
57     android:paddingRight="10dp"
58     android:visibility="gone"
59 />
60
61 <ProgressBar
62     android:layout_width="wrap_content"
63     android:layout_height="wrap_content"
64     android:id="@+id/progressBar"
65     android:indeterminate="true"
66     android:visibility="visible"
67     android:layout_gravity="center"
68     android:layout_weight="1"/>
69 </LinearLayout>
70
71 <ListView
72     android:layout_width="fill_parent"
73     android:layout_height="0dp"
74     android:id="@+id/main_listView"
75     android:choiceMode="singleChoice"
76     android:layout_alignParentBottom="true"
77     android:layout_alignParentLeft="true"
78     android:entries="@array/main_list_items"
79     android:divider="@android:drawable/divider_horizontal_dark"
80     android:layout_weight="1"
81     tools:listitem="@android:layout/simple_list_item_1"
82     android:layout_gravity="bottom"
83     android:stackFromBottom="true"
84 />
85
86 </LinearLayout>
87
88
89
90
91
92 <!--?xml version="1.0" encoding="utf-8"?-->
93
94 <!--RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"-->
95 <!--xmlns:tools="http://schemas.android.com/tools"-->
96 <!--android:orientation="vertical"-->
97 <!--android:layout_width="fill_parent"-->
98 <!--android:layout_height="fill_parent"-->

```



```

99      <!--android:gravity="fill_horizontal"-->
100
101      <!--<TextView-->
102      <!--android:layout_width="wrap_content"-->
103      <!--android:layout_height="wrap_content"-->
104      <!--android:textAppearance="?android:attr/textAppearanceLarge"-->
105      <!--android:text="This week at Journey"-->
106      <!--android:id="@+id/main_title_textview"-->
107      <!--android:layout_gravity="center"-->
108      <!--android:gravity="center" android:padding="10dp"-->
109      <!--android:layout_alignParentTop="true"-->
110      <!--android:layout_alignParentLeft="true"-->
111      <!--android:layout_alignParentRight="true"/>-->
112
113      <!--<TextView-->
114      <!--android:layout_width="wrap_content"-->
115      <!--android:layout_height="wrap_content"-->
116      <!--android:textAppearance="?android:attr/textAppearanceMedium"-->
117      <!--android:id="@+id/main_sermon_title_textView" android:layout_gravity="center" android:textStyle
="bold"-->
118      <!--android:gravity="center" android:padding="10dp"-->
119      <!--android:text="Welcome"-->
120      <!--android:layout_above="@+id/main_listView"-->
121      <!--android:layout_centerHorizontal="true"/>-->
122
123      <!--<ImageView-->
124      <!--android:layout_width="match_parent"-->
125      <!--android:layout_height="wrap_content"-->
126      <!--android:id="@+id/imageView"-->
127      <!--android:src="@drawable/journey_logo"-->
128      <!--android:scaleType="fitCenter"-->
129      <!--android:padding="10dp"-->
130      <!--android:layout_below="@+id/main_title_textview"-->
131      <!--android:layout_above="@+id/main_sermon_title_textView"-->
132      <!--android:layout_centerHorizontal="true"/>-->
133
134      <!--<ListView-->
135      <!--android:layout_width="wrap_content"-->
136      <!--android:layout_height="wrap_content"-->
137      <!--android:id="@+id/main_listView"-->
138      <!--android:choiceMode="singleChoice"-->
139      <!--android:layout_alignParentBottom="true"-->
140      <!--android:layout_alignParentLeft="true"-->
141      <!--android:entries="@array/main_list_items"-->
142      <!--android:divider="@android:drawable/divider_horizontal_dark"/>-->
143
144      <!--</RelativeLayout-->

```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_height="wrap_content"
4     android:layout_width="fill_parent"
5     android:padding="6dip"
6     android:orientation="vertical"
7     android:background="?android:attr/activatedBackgroundIndicator">
8
9     <ImageView
10         android:id="@+id/sermon_thumbnail_imageview"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:layout_alignParentLeft="true"
14         android:layout_centerVertical="true"
15         android:layout_marginRight="6dip"
16         android:src="@drawable/ic_launcher_journey"
17         android:maxHeight="200dp"/>
18
19     <TextView
20         android:id="@+id/sermon_title_textview"
21         android:layout_width="fill_parent"
22         android:layout_height="wrap_content"
23         android:layout_toRightOf="@id/sermon_thumbnail_imageview"
24         android:maxLines="2"
25         android:overScrollMode="never"
26         android:scrollHorizontally="true"
27         android:textSize="15sp"
28         android:textStyle="bold" />
29
30     <TextView
31         android:id="@+id/sermon_date_textview"
32         android:autoLink="all"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:layout_alignParentRight="true"
36         android:layout_below="@id/sermon_title_textview"
37         android:layout_toRightOf="@id/sermon_thumbnail_imageview"
38         android:textSize="15sp" />
39
40 </RelativeLayout>
41
```

```

1 <fragment xmlns:android="http://schemas.android.com/apk/res/android"
2   xmlns:tools="http://schemas.android.com/tools"
3   android:id="@+id/sermon_list"
4   android:name="com.chris.churchapp.fragments.SermonListFragment"
5   android:layout_width="match_parent"
6   android:layout_height="match_parent"
7   tools:context=".SermonListActivity"
8   tools:layout="@android:layout/sermon_list_item" />
9   <!--android:layout_marginLeft="16dp"-->
10  <!--android:layout_marginRight="16dp"-->
11
12 <!--<?xml version="1.0" encoding="utf-8"?-->
13 <!--&lt;!&ndash; Copyright (C) 2011 The Android Open Source Project-->
14
15 <!--Licensed under the Apache License, Version 2.0 (the "License");-->
16 <!--you may not use this file except in compliance with the License.-->
17 <!--You may obtain a copy of the License at-->
18
19 <!--http://www.apache.org/licenses/LICENSE-2.0-->
20
21 <!--Unless required by applicable law or agreed to in writing, software-->
22 <!--distributed under the License is distributed on an "AS IS" BASIS.-->
23 <!--WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.-->
24 <!--See the License for the specific language governing permissions and-->
25 <!--limitations under the License.-->
26 <!--&ndash;&gt;-->
27
28 <!--&lt;!&ndash; BEGIN_INCLUDE(all) &ndash;&gt;-->
29 <!--<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"-->
30 <!--xmlns:tools="http://schemas.android.com/tools"-->
31 <!--android:orientation="vertical"-->
32 <!--android:layout_width="match_parent"-->
33 <!--android:layout_height="match_parent"-->
34
35 <!--<fragment android:id="@+id/sermon_list"-->
36 <!--android:layout_height="fill_parent"-->
37 <!--android:name="com.chris.churchapp.fragments.SermonListFragment"-->
38 <!--android:layout_width="match_parent"-->
39 <!--tools:layout="@layout/sermon_list_item"/>-->
40 <!--</LinearLayout-->-->
41 <!--&lt;!&ndash; END_INCLUDE(all) &ndash;&gt;-->
42

```

```
1 <FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
2   xmlns:tools="http://schemas.android.com/tools"
3   android:id="@+id/sermon_detail_container"
4   android:layout_width="match_parent"
5   android:layout_height="match_parent"
6   tools:context=".SermonDetailActivity"
7   tools:ignore="MergeRootFrame" />
8
```

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical"
7     android:id="@+id/sermon_detail"
8     tools:context=".SermonDetailFragment"
9 >
10
11 <LinearLayout
12     android:layout_width="match_parent"
13     android:layout_height="wrap_content"
14     android:gravity="center"
15     android:orientation="vertical" >
16
17     <TextView
18         android:id="@+id/sermon_detail_title_textView"
19         android:layout_width="wrap_content"
20         android:layout_height="wrap_content"
21         android:gravity="center_horizontal"
22         android:text="test"
23         android:textAppearance="?android:attr/textAppearanceMedium"
24         android:paddingTop="10dp" />
25         <!--android:paddingBottom="10dp" />-->
26
27 </LinearLayout>
28
29 <LinearLayout
30     android:layout_width="match_parent"
31     android:layout_height="0dp"
32     android:layout_weight="2"
33     android:gravity="center"
34     android:orientation="vertical" >
35
36     <ImageView
37         android:id="@+id/sermon_detail_imageView"
38         android:layout_width="fill_parent"
39         android:layout_height="fill_parent"
40         android:scaleType="fitCenter"
41         android:src="@drawable/journey_logo"
42         android:padding="10dp" />
43
44 </LinearLayout>
45
46 <LinearLayout
47     android:layout_width="match_parent"
48     android:layout_height="0dp"
49     android:layout_weight="1"
```